

KAIF — Krinik AI Framework

License **MIT** Version **1.0** Core **Self-extracting** For **AI coding agents** Docs **EN | RU**

A context-resilient, autonomy-disciplined operating framework for AI coding agents. Drop it into any project — in any domain — to turn your AI agent (Claude or any other) into a disciplined, autonomous teammate that never starts from zero. The human stays the visionary; the agent executes. KAIF is the methodology binding them — with a full lifecycle: deploy → update → fork → remove.

🇬🇧 English (primary) below · 🇷🇺 [Русская версия ниже](#) 📦 The whole framework is one file: [FRAMEWORK.md](#) — a self-extracting core.

Why

AI coding agents are powerful but suffer two chronic failures:

- **They forget.** Context is lost between sessions. Every new session re-discovers the architecture, the decisions, the half-finished work, the bug it was chasing yesterday.
- **They drift.** Left autonomous, an agent either stalls (over-engineering a misunderstood task) or oversteps (making brand/architecture decisions that weren't its to make).

KAIF fixes both by **externalizing the agent's working memory and discipline into the repository itself** — a small set of markdown files, directory conventions, and repeatable slash-skills. The result: any fresh session resumes instantly with full context, works autonomously within clear bounds, and accumulates knowledge instead of evaporating it.

It is **not code** — it is *process, captured as files an agent reads*. It works with any language, any stack, any project. It was distilled from the real working method that emerged as Krinik and Claude built software together — a standalone by-product of that collaboration, generalized for everyone.

How it works

You (human)

Your AI agent

1. Drop `FRAMEWORK.md` into your repo
2. "Unpack the KAIF framework" → reads `FRAMEWORK.md`, inspects your project, writes out the structure, customizes it
3. `/resume` · `/pause` · `/autoloop` ... → works like a disciplined, autonomous teammate

Your language, your project

The framework's sources are English (the community language), but **on deploy the agent asks your preferred working language and writes your project's working docs and skills in it** — unpacking from the English sources into whatever language suits your team. Code, commands, and `/command` names stay canonical; the prose speaks your language. (*This repo is the example: an English payload with a Russian working wrapper.*)

Quick start (for the human)

1. **Get the framework.** Download [FRAMEWORK.md](#) into your project's root — or clone this repo alongside your project:

```
git clone https://github.com/MikalaiKryvusha/KAIF.git
```

2. **Ask your agent to unpack it.** In your AI coding agent, open your project and say:

"Read `FRAMEWORK.md` and unpack the KAIF framework into this project."

The agent inspects your project, creates the structure, fills in the project-specific details (build command, architecture, etc.), asks your **preferred working language** (default English), and may ask a few owner-level questions (brand, license) via a short interview document — answer them.

3. Drive it with skills:

- `/resume` — start a session (the agent reads the state and proposes the next move).
- `/pause` — wrap up (record state, build, commit, push).
- `/autoloop` · `/dayloop` · `/nightloop` — autonomous work while you're away.
- `/report-bug` · `/propose-idea` · `/interview` · `/release` — as the names suggest.

What gets unpacked

```
your-project/
├─ AGENT_GUIDE.md           # THE canon — read before every task (rules, map, commands, conventions)
├─ PHILOSOPHY.md           # how the agent thinks: KISS + Occam (simplicity above cleverness)
├─ BUG_FIXING_FRAMEWORK.md  # how the agent debugs: the fix→build→test loop, the 3-attempts rule
├─ STATUS.md               # the living state — updated after every significant task
├─ plans/                  # master_plan.md (roadmap) · project_map.md (architecture) · ideas/
├─ bugs/                   # one doc per defect (symptom → forensics → root cause → fix)
├─ interviews/             # A/B/C questions for the human on owner-level decisions
└─ .claude/skills/         # the twelve repeatable rituals (below)
```

The skills

Skill	Purpose
<code>/resume</code>	Start a session — read the docs, pick the one main thing, begin.
<code>/pause</code>	End a session — update STATUS & README, build, commit, push.
<code>/autoloop</code>	Grind a pool of autonomous tasks in a self-restarting series.
<code>/dayloop</code>	Continuous autonomous work while you're busy (no time stop).
<code>/nightloop</code>	Autonomous work until a wake time / you return.
<code>/refresh-context</code>	Re-read the plan & guidance; rebuild the backlog.
<code>/check-backlog</code>	Tag finished work DONE; return the open list.
<code>/report-bug</code>	File a bug document by the canon.
<code>/bug-research</code>	Deep investigation (no coding) after 3 failed fix attempts.
<code>/propose-idea</code>	Propose a feature for your approval.
<code>/interview</code>	Ask you closed A/B/C questions on a fateful decision.
<code>/release</code>	Publish a release to GitHub (with confirmation; never autonomously).
<code>/kaif-version</code> · <code>/kaif-update</code>	Report the deployed version; respectfully update & migrate from origin.
<code>/kaif-fork</code> · <code>/kaif-switch-origin</code>	Snapshot KAIF to your own repo; or switch tracking back to origin.
<code>/kaif-remove</code>	Respectfully remove KAIF — partial (keep artifacts) or full; your project stays intact.

Lifecycle, any domain, any agent

- **A lifecycle, not a one-shot install.** KAIF is versioned (`vX.Y.Z` (DATE)), recorded in `.kaif/kaif.json` . It can **update & migrate** from origin respectfully, **fork** into your own repo to evolve independently, switch back, and be

respectfully removed — keeping your bug reports, interviews, and ideas, or fully, leaving your own project untouched. Backed by `npm run kaif:*` handles.

- **Any domain, not just code.** A *sphere* (programming, science, design, business, ...) tailors the terminology at deploy time via per-sphere term libraries — so KAIF fits research, design, PM, finance, writing, and more.
- **Any agent, not just Claude.** Adapters wire KAIF into each system's conventions (Claude Code, OpenAI Codex, GitHub Copilot, Cursor, Windsurf, Cline, Roo Code, ...), always with a universal `AGENTS.md` fallback.

Four ideas hold it together

1. **Externalized memory** — the agent's state lives in files, not the chat. A fresh session is instantly productive.
2. **Knowledge that accumulates** — bugs, decisions, and proposals become durable documents, not lost chat.
3. **Bounded autonomy** — the agent decides what's cheap to reverse; it escalates brand/UX/architecture via interviews.
4. **Simplicity above cleverness** — a stall means you misunderstood the task, not that it's hard. KISS + Occam.

For AI agents

If you are an AI agent: read [FRAMEWORK.md](#) end to end, then follow its *"Unpacking — step by step"* section. Everything you need is inside that one document.

This repository is fractal (dogfooding)

This repo *is* the framework **and** is *wrapped by* the framework — it uses itself. Its root holds a real `AGENT_GUIDE.md`, `STATUS.md`, `.claude/skills/`, `plans/`, `bugs/`, `interviews/` describing the development of the framework itself.

⚠ **When you deploy the framework into your project, derive everything from `FRAMEWORK.md` only** — not from this repo's own wrapper files (they're about building the framework, not your project). `FRAMEWORK.md` is the single source of truth for deployment. It is **generated** from `framework/` by `node tools/build-framework.mjs` — never hand-edit it.

Repository layout

FRAMEWORK.md / FRAMEWORK.ru.md	★ the self-extracting core (EN / RU), generated
framework/	the canonical universal templates (the payload)
tools/	build-framework.mjs · readme-pdf.mjs · commit.mjs
README.md / README.pdf	this front door (EN+RU) and its rendered copy
goal.md	the original vision
AGENT_GUIDE.md PHILOSOPHY.md ...	the dogfooding wrapper (the framework on itself)

License

[MIT](#) — © 2026 Mikalai Kryvusha aka **KOT KRINIK** · Николай Кривуша aka Кот Криник.

Use it, copy it, modify it, ship it — including, as this repo shows, on the framework's own project.

KAIF — Фреймворк для ИИ-агента (на русском)

License **MIT** Version **1.0** Ядро **Самораспаковывающееся**

Фреймворк работы для ИИ-агентов-программистов: устойчивость к потере контекста и дисциплина автономности. Положите его в любой проект — в любой сфере — и ваш ИИ-агент (Claude или любой другой) превратится в дисциплинированного автономного напарника, который никогда не начинает с нуля. Человек остаётся визионером, агент — исполнителем; KAIF — методология, их связывающая, с полным жизненным циклом: развёртывание → обновление → форк → удаление.

📦 Весь фреймворк — это один файл: [FRAMEWORK.ru.md](https://github.com/MikalaiKryvusha/FRAMEWORK.md) — самораспаковывающееся ядро (английский оригинал: [FRAMEWORK.md](https://github.com/MikalaiKryvusha/FRAMEWORK.md)).

Зачем

ИИ-агенты-программисты мощны, но страдают двумя хроническими бедами:

- **Они забывают.** Контекст теряется между сессиями. Каждая новая сессия заново выясняет архитектуру, принятые решения, недоделанную работу, баг, который ловили вчера.
- **Их «уводит».** Будучи автономным, агент либо застревает (переусложняя неверно понятую задачу), либо превышает полномочия (принимая решения о бренде/архитектуре, которые принимать был не вправе).

KAIF лечит и то и другое, **вынося рабочую память и дисциплину агента в сам репозиторий** — в виде небольшого набора markdown-файлов, соглашений о директориях и повторяемых /слеш-навыков. Итог: любая новая сессия мгновенно включается в работу с полным контекстом, работает автономно в чётких границах и накапливает знания, а не испаряет их.

Это **не код** — это *процесс, зафиксированный как файлы, которые читает агент*. Он работает с любым языком, любым стеком, любым проектом. Фреймворк выделен («дистиллирован») из реального метода работы, сложившегося у Криника в паре с Claude в совместной разработке софта, — самостоятельный побочный продукт этого сотрудничества, обобщённый для всех.

Как это работает

Вы (человек)	Ваш ИИ-агент
1. Кладёте FRAMEWORK.md в свой репозиторий	
2. «Распакуй фреймворк KAIF»	→ читает FRAMEWORK.md, изучает проект, разворачивает структуру, настраивает её
3. /resume · /pause · /autoloop ...	→ работает как дисциплинированный автономный напарник

Ваш язык — ваш проект

Исходники фреймворка — на английском (язык сообщества), но **при распаковке агент спрашивает ваш предпочитаемый рабочий язык и пишет рабочие документы и навыки вашего проекта на нём** — разворачивая из английских исходников в язык, удобный вашей команде. Код, команды и имена /команд остаются каноническими; на ваш язык переводится текст. (Этот репозиторий — пример: английская полезная нагрузка и русская рабочая обвязка.)

Быстрый старт (для человека)

1. **Возьмите фреймворк.** Скачайте [FRAMEWORK.md](https://github.com/MikalaiKryvusha/FRAMEWORK.md) в корень своего проекта — или склонируйте этот репозиторий рядом:

```
git clone https://github.com/MikalaiKryvusha/KAIF.git
```

2. **Попросите агента распаковать.** В своём ИИ-агенте откройте проект и скажите:

```
«Прочитай FRAMEWORK.md и распакуй фреймворк KAIF в этот проект.»
```

Агент изучит проект, создаст структуру, заполнит специфику проекта (команда сборки, архитектура и т.д.) спросит ваш **предпочитаемый рабочий естественный язык, на котором вы разговариваете в повседневной жизни** (по умолчанию английский) и может задать несколько вопросов уровня владельца (бренд, лицензия) через короткий документ-интервью — ответьте на них.

3. **Управляйте навыками:**

- `/resume` — начать сессию (агент читает состояние и предлагает следующий шаг).
- `/pause` — завершить (зафиксировать статус, собрать, коммитить, запустить).

- o /autoloop · /dayloop · /nightloop — автономная работа, пока вас нет.
- o /report-bug · /propose-idea · /interview · /release — по названию.

Что распаковывается

ваш-проект/	
— AGENT_GUIDE.md	# КАНОН — читать перед каждой задачей (правила, карта, команды, соглашения)
— PHILOSOPHY.md	# как агент мыслит: KISS + Бритва Оккама (простота важнее «умности»)
— BUG_FIXING_FRAMEWORK.md	# как агент чинит баги: цикл фикс→сборка→тест, правило 3 попыток
— STATUS.md	# живое состояние — обновляется после каждой значимой задачи
— plans/	# master_plan.md (генплан) · project_map.md (архитектура) · ideas/
— bugs/	# по документу на дефект (симптом → форензика → причина → фикс)
— interviews/	# вопросы A/B/C к человеку по решениям уровня владельца
— .claude/skills/	# двенадцать повторяемых ритуалов (ниже)

Навыки

Навык	Назначение
/resume	Начать сессию — прочитайте доки, выберите главное, приступите.
/pause	Завершить сессию — обновите STATUS и README, соберите, закоммитьте, запустите.
/autoloop	Грайндить пул автономных задач серией с самоперезапуском.
/dayloop	Непрерывная автономная работа, пока вы заняты (без стопа по времени).
/nightloop	Автономная работа до времени пробуждения / вашего возвращения.
/refresh-context	Перечитать план и руководства; пересобрать беклог.
/check-backlog	Пометить сделанное DONE; вернуть список открытого.
/report-bug	Завести баг-документ по канону.
/bug-research	Глубокое исследование (без кода) после 3 неудачных попыток фикса.
/propose-idea	Предложить фичу на ваше одобрение.
/interview	Задать вам закрытые вопросы A/B/C по судьбоносному решению.
/release	Выпустить релиз в GitHub (с подтверждением; никогда автономно).
/kaif-version · /kaif-update	Показать версию; уважительно обновиться и мигрировать из origin.
/kaif-fork · /kaif-switch-origin	Слепок KAIF в свой репозиторий; или переключиться обратно на origin.
/kaif-remove	Уважительно удалить KAIF — частично (с сохранением артефактов) или полностью.

Жизненный цикл, любая сфера, любой агент

- **Жизненный цикл, а не разовая установка.** KAIF версионизируется (vX.Y.Z (DATA)) и фиксируется в .kaif/kaif.json . Он умеет **уважительно обновляться** и **мигрировать** из origin, делать **форк** в ваш репозиторий для независимой эволюции, переключаться обратно и **уважительно удаляться** — с сохранением ваших баг-репортов, интервью и идей, или полностью, не трогая ваш проект. Через «ручки» npm run kaif:* .

- **Любая сфера, не только код.** Сфера (программирование, наука, дизайн, бизнес, ...) настраивает терминологию при развёртывании через библиотеки терминов — KAIF подходит для исследований, дизайна, проджект-менеджмента, финансов, писательства и прочего.
- **Любой агент, не только Claude.** Адаптеры подключают KAIF к соглашениям каждой системы (Claude Code, OpenAI Codex, GitHub Copilot, Cursor, Windsurf, Cline, Roo Code, ...), всегда с запасным `AGENTS.md`.

Четыре идеи, на которых всё держится

1. **Вынесенная память** — состояние агента живёт в файлах, а не в чате. Новая сессия продуктивна сразу.
2. **Накопление знаний** — баги, решения и идеи становятся долговечными документами, а не теряются в чате.
3. **Ограниченная автономность** — агент сам решает то, что легко откатить; бренд/UX/архитектуру выносит на интервью.
4. **Простота важнее «умности»** — затык значит, что вы не поняли задачу, а не что она сложна. KISS + Оккам.

Для ИИ-агентов

Если вы — ИИ-агент: прочитайте [FRAMEWORK.ru.md](#) (или оригинал [FRAMEWORK.md](#)) целиком, затем следуйте разделу «Распаковка — по шагам». Всё нужное — внутри этого одного документа.

Этот репозиторий фрактален (самообвязка)

Этот репозиторий является фреймворком и *обязан* фреймворком — он использует сам себя. В его корне лежат настоящие `AGENT_GUIDE.md`, `STATUS.md`, `.claude/skills/`, `plans/`, `bugs/`, `interviews/`, описывающие разработку самого фреймворка.

⚠ **Разворачивая фреймворк в свой проект, берите всё ТОЛЬКО из `FRAMEWORK.md`** — а не из собственных обязательных файлов этого репозитория (они про разработку фреймворка, а не вашего проекта). `FRAMEWORK.md` — единственный источник истины для развёртывания. Он **генерируется** из `framework/` командой `node tools/build-framework.mjs` — никогда не правьте его руками.

Структура репозитория

FRAMEWORK.md / FRAMEWORK.ru.md	★ самораспаковывающееся ядро (EN / RU), генерируется
framework/	канонические универсальные шаблоны (полезная нагрузка)
tools/	<code>build-framework.mjs</code> · <code>readme-pdf.mjs</code> · <code>commit.mjs</code>
README.md / README.pdf	этот «парадный вход» (EN+RU) и его рендер-копия
goal.md	исходное видение
AGENT_GUIDE.md PHILOSOPHY.md ...	обязка-самообёртка (фреймворк, применённый к себе)

Лицензия

[MIT](#) — © 2026 **Mikalai Kryvusha** aka **KOT KRINIK** · Николай Кривуша aka Кот Криник.

Используйте, копируйте, изменяйте, распространяйте — в том числе, как показывает этот репозиторий, на проекте самого фреймворка.